

Sketch 1: Situational English Practice Tool for International Students

Project Description

For this project, I would like to create a simple English practice tool for international students. Many international students have learned English vocabulary and grammar, but speaking English in real-life situations can still be difficult. Sometimes they know what they want to say, but they are not sure how to say it naturally. They may also feel nervous when they need to start or respond to a conversation.

The program would let users choose a common situation they may experience in daily life, such as going to the post office, ordering food at a restaurant, buying coffee, shopping at a grocery store, or talking with someone at school. After choosing a situation, the user would see some useful phrases and a short example conversation. They could then practice the situation by choosing appropriate responses.

Who would be the users? Are there secondary stakeholders?

The primary users would be international students who want to feel more comfortable speaking English in everyday situations. It could be especially helpful for students who recently came to the United States and are still getting used to communicating in English.

Other English learners who want to practice everyday conversations could also use the tool.

What problem would it help to solve?

International students may understand English in class but still have difficulty speaking in real life situations. One reason is that they may not know what the other person is going to say or how they should respond.

This tool would give students a way to prepare before entering a real situation. For example, before going to a post office, a student could review common expressions for mailing a package and practice a short conversation.

What is the workflow (user path)? What would the user do? What is the primary interaction? Is there an interaction loop?

The user would first choose a situation, such as Post Office, and then choose a specific task, such as Mail a Package. The program would show several useful phrases for that task.

After reviewing the phrases, the user could start a short practice conversation. The program would show what another person might say, and the user would choose a response from several options. The conversation would continue for a few steps.

Choose a situation → Choose a task → Review useful phrases → Practice a short conversation

After finishing, the user could return to the menu and practice another situation.

What data would be used, how would you get it, and how would it be processed/analyzed?

I would create a small collection of common English phrases and sample conversations for different situations. The data could be stored in a CSV or JSON file. Each item could include the situation, task, English phrase, Chinese meaning, and conversation step.

Python would read the data and filter the phrases based on the situation and task selected by the user. For the practice conversation, conditional statements could be used to determine what happens after the user chooses a response.

What are the results and how are they presented?

The results would be presented through a simple interface. The user would first see useful phrases for the selected situation and then have the option to practice a short conversation.

For example:

Situation: Post Office

Task: Mail a Package

I'd like to mail this package.
How much does it cost to send this?
How long will it take to arrive?
Does this include tracking?

The practice section would then show a short conversation and several possible responses.

What would be the biggest problem to implement it as is?

One challenge would be organizing the phrases and conversations so that they sound natural and are useful in different situations. I would also need to make sure the conversation moves correctly from one step to another.

To keep the project manageable, the first version would only include a limited number of common situations and short, predefined conversations. I would not include speech recognition or AI-generated conversations.

What part can I do myself, on my own or with some help?

I think I could create and organize the phrase data and write the basic Python logic using lists, dictionaries, loops, functions, and conditional statements. I may need some help with reading data from a CSV or JSON file and building a simple user interface.

Sketch 2: Recipe Finder from Available Ingredients

Project Description

For this project, I would like to create a recipe finder that helps users decide what they can cook with ingredients they already have at home. Sometimes people have food in their refrigerator or kitchen but are not sure what they can make with it. Some ingredients may eventually go bad if they are not used.

The user would enter or select ingredients they currently have, such as eggs, tomatoes, chicken, rice, or potatoes. The program would compare those ingredients with a small collection of recipes and find the best matches. It would also show which ingredients the user already has and which ones are still missing.

Who would be the users? Are there secondary stakeholders?

The main users would be students or other people who cook at home and want an easy way to decide what to make with available ingredients. It could be especially useful for people who have limited ingredients, want to save money, or want to reduce food waste.

Anyone who wants ideas for using leftover ingredients could also use the program.

What problem would it help to solve?

People sometimes already have food at home but do not know what they can make with it. They may end up buying more food, ordering takeout, or letting some ingredients go unused.

Instead of asking the user to search for a specific recipe, this program would start with the ingredients the user already has and suggest possible recipes.

What is the workflow (user path)? What would the user do? What is the primary interaction? Is there an interaction loop?

The user would enter or select the ingredients they have. The program would compare their ingredients with the ingredients needed for each recipe and calculate which recipes are the closest matches.

The user would then see several recommendations and could select one recipe to see more details.

Enter ingredients → Find matches → View recommended recipes → Select a recipe → View available and missing ingredients

If the user adds or removes an ingredient, they could search again and get new recommendations.

What data would be used, how would you get it, and how would it be processed/analyzed?

For the first version, I would create a small recipe dataset and store it in a CSV or JSON file. Each recipe could contain a recipe name, required ingredients, and simple cooking instructions.

Python would compare the user's ingredient list with the ingredients required for each recipe. It could calculate the percentage of ingredients the user already has and then sort the recipes from the highest match to the lowest match.

For example, if a recipe needs five ingredients and the user has four of them, the result could show an **80% match**.

Using a small local dataset would also mean that the first version would not depend on an external recipe API.

What are the results and how are they presented?

The program would present a list of recommended recipes, starting with the recipes that best match the user's available ingredients.

For example:

You have:

Eggs, tomatoes, rice, green onions

Recommended Recipe: Tomato and Egg Stir-Fry

You already have:

- Eggs
- Tomatoes
- Green onions

Missing:

- Cooking oil

The program could also display a match percentage so the user can compare different recipes.

What would be the biggest problem to implement it as is?

One challenge would be handling ingredients that have slightly different names or forms. For example, "tomato" and "tomatoes" should probably be recognized as the same ingredient.

Another challenge would be deciding how many recipes to include. To keep the project realistic, I would start with a small recipe collection instead of trying to create a very large recipe database.

What part can I do myself, on my own or with some help?

I think I could create the recipe data and write the basic ingredient matching logic using lists, dictionaries, loops, and functions. I may need some help calculating and sorting the match results and creating a simple interface for entering ingredients and viewing recipes.

Sketch 3: Weather-Based Outfit Advisor

Project Description

For this project, I would like to create a simple tool that recommends what to wear based on the weather. Checking the temperature does not always make it easy to decide what clothes to wear, especially when there is also rain, snow, or strong wind.

The user would enter a location, and Python would get current or forecast weather information such as temperature, precipitation, and wind. The program would use this information to give simple clothing suggestions, such as wearing a jacket, wearing long pants, or bringing an umbrella.

Who would be the users? Are there secondary stakeholders?

The main users would be students or other people who want a quick suggestion about what to wear before going outside.

It could be especially useful for international students or people who recently moved to an area with weather conditions they are not familiar with.

What problem would it help to solve?

A weather app can tell someone that it is 45°F outside, but that person may still be unsure about what they should wear. For someone who is not familiar with the local climate, it may be difficult to know whether a light jacket is enough or whether a heavier coat is needed.

Weather conditions such as rain and wind can also affect what feels comfortable. This program would turn basic weather information into a simple, practical clothing suggestion.

What is the workflow (user path)? What would the user do? What is the primary interaction? Is there an interaction loop?

The user would enter or select a location. The program would retrieve weather information for that location and check conditions such as temperature, rain, snow, and wind.

It would then display a clothing recommendation.

Enter location → Get weather data → Check weather conditions → Show clothing suggestions

The user could then enter another location or check the weather again later.

What data would be used, how would you get it, and how would it be processed/analyzed?

The program would use weather information from a free weather API. The data could include temperature, precipitation, weather conditions, and wind speed.

Python would request the weather data and read the information returned in JSON format. Conditional statements would then be used to determine the clothing recommendation.

For example:

- Very cold → heavy coat
- Cool → jacket

- Rain → umbrella or rain jacket
- Strong wind → extra layer

For the final project, I would need to choose a free weather API and confirm what weather data it provides before implementing this part.

What are the results and how are they presented?

The program would show a short weather summary followed by a clothing recommendation.

For example:

Weather

45°F

Windy

Light rain

Suggestion

Wear a jacket and long pants.

Bring an umbrella or a rain jacket.

The information could be displayed in a simple desktop or web interface.

What would be the biggest problem to implement it as is?

The main challenge would be learning how to request data from a weather API and correctly use the returned information in Python. I would also need to decide how different combinations of temperature, rain, wind, and snow should affect the clothing recommendation.

To keep the project manageable, the first version would use simple rules and a limited number of weather conditions.

What part can I do myself, on my own or with some help?

I think I could write the basic clothing recommendation rules using `if`, `elif`, and `else` statements. I would probably need some help learning how to request weather data from an API, work with JSON data, and connect the weather information to a simple user interface.